# Performance Benchmark of Elliptic Curve Cryptography on MicroBlaze Processor

Tommy Duong, Sikan Fu, Torin Lee, Denny Nguyen
*Electrical and Computer Engineering Department*
*California State Polytechnic University, Pomona*

***Abstract*—**Elliptic Curve Cryptography (ECC) is a modern public-key encryption technique that provides strong security with relatively small key sizes, making it well-suited for embedded systems and resource-constrained environments. This project benchmarks the performance of ECC algorithms implemented on a MicroBlaze softcore processor running on a Nexys A7-100T FPGA. Specifically, the secp160r1, secp192r1, and secp256r1 curves were evaluated. Performance metrics including latency and throughput were measured for key generation, ECDSA signing, and verification. The results demonstrate that increasing key size significantly impacts execution time and system efficiency, highlighting the trade-offs between security strength and performance in embedded applications.

*Index Terms*—Elliptic Curve Cryptography (ECC), MicroBlaze, FPGA, Benchmarking, Embedded Systems

## I. INTRODUCTION

The increasing use of embedded systems and Internet of Things (IoT) devices has created a growing need for efficient and secure cryptographic solutions. These systems often operate under strict constraints in terms of processing power, memory, and energy, making it difficult to implement traditional encryption algorithms. Public-key cryptography is essential for secure communication, but widely used methods such as RSA require large key sizes and high computational cost, which are not ideal for resource-constrained environments.

Elliptic Curve Cryptography (ECC) offers an alternative approach by providing equivalent security with significantly smaller key sizes. This allows for reduced memory usage and improved computational efficiency, making ECC well-suited for embedded applications. However, ECC operations such as key generation, signing, and verification are still computationally intensive, especially as key sizes increase. This introduces a trade-off between stronger security and system performance.

This project presents a benchmarking study of ECC algorithms implemented on a MicroBlaze softcore processor running on a Nexys A-7 FPGA. The secp160r1, secp192r1, and secp256r1 curves were evaluated to analyze how increasing key size affects execution time and throughput. By maintaining a constant hardware configuration, this study isolates the performance impact of stronger cryptographic parameters in embedded systems.

## II. BACKGROUND

Elliptic Curve Cryptography (ECC) is a public-key cryptographic technique based on the mathematical properties of elliptic curves over finite fields. An elliptic curve is typically defined by the equation:

$$y^2 = x^3 + ax + b$$

The security of ECC relies on the difficulty of the Elliptic Curve Discrete Logarithm Problem (ECDLP), which makes it computationally infeasible to determine a private key from a public key. Compared to traditional methods such as RSA, ECC achieves the same level of security using much smaller key sizes, resulting in reduced memory usage and faster computations, making it well-suited for embedded systems and IoT devices [1].

ECC operations, including key generation, signing, and verification, depend on point addition and scalar multiplication. Although ECC is more efficient than RSA, these operations remain computationally intensive, especially on softcore processors. As key size increases, execution time also increases, creating a trade-off between security and performance that must be considered in embedded system design [2].

Standardized curves such as secp160r1, secp192r1, and secp256r1 are widely used [3] and provide increasing levels of security. However, higher security levels require more computation time and system resources, making performance evaluation critical for embedded implementations [2].

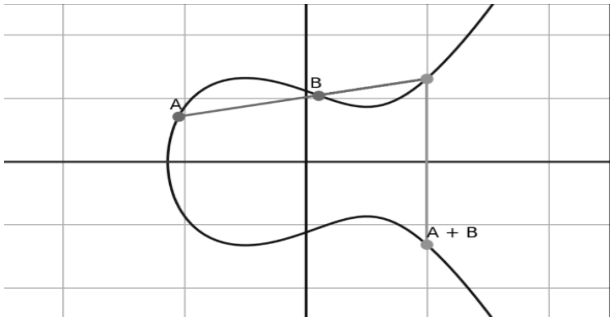


Fig. 1. Elliptic curve point addition used in ECC.

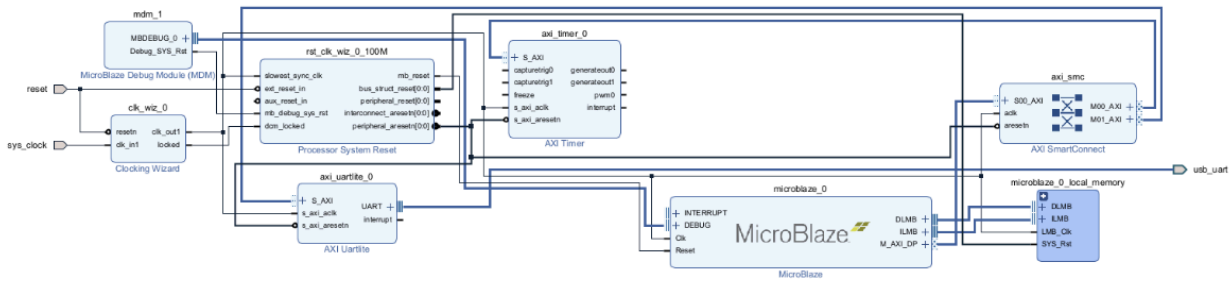


*Fig. 2. MicroBlaze processor system architecture.*

## III. METHODOLOGY

### *A. Hardware Setup*

The system was implemented on a Digilent Nexys A7-100T FPGA using a MicroBlaze softcore processor [4]. The design includes a clocking module, AXI timer for measuring execution cycles, and UART for output display.

### *B. Software Implementation*

ECC operations were implemented using a lightweight cryptographic library. A benchmarking function was used to measure execution cycles for key generation, signing, and verification using the AXI timer [5].

### *C. Test Configuration*

Three ECC curves were tested:

- secp160r1 (160-bit)
- secp192r1 (192-bit)
- secp256r1 (256-bit)

Metrics collected:

- Cycle count
- Execution time
- Throughput

## IV. RESULTS AND ANALYSIS

The performance results for ECC operations across different curve sizes were obtained using the MicroBlaze processor under a consistent hardware configuration. By maintaining the same system setup, any differences in performance can be directly attributed to the computational complexity of each elliptic curve.

### *A. Performance Scaling with Curve Size*

From the collected results, it can be observed that ECC performance is strongly dependent on the curve size. As the bit size increases from secp160r1 to secp256r1, both the cycle count and execution time increase significantly across all operations.

In order to clearly present these results, the measured performance metrics are summarized in Table I.

TABLE I
ECC PERFORMANCE METRICS BY CURVE SIZE

| Metric | secp160r1 | secp192r1 | secp256r1 |
|---|---|---|---|
| KeyGen Cycles | 60,126,979 | 94,129,079 | 234,176,849 |
| Signing Cycles | 62,511,860 | 96,369,492 | 238,078,119 |
| Verify Cycles | 70,205,010 | 108,902,654 | 267,640,482 |
| KeyGen Time (s) | 0.6 | 0.94 | 2.34 |
| Signing Time (s) | 0.62 | 0.96 | 2.38 |
| Verify Time (s) | 0.7 | 1.09 | 2.68 |
| KeyGen Throughput | 1.67 | 1.06 | 0.43 |
| Signing Throughput | 1.61 | 1.04 | 0.42 |
| Verify Throughput | 1.43 | 0.92 | 0.37 |

As shown in Table I, key generation increases from approximately 60 million cycles for secp160r1 to over 234 million cycles for secp256r1, representing nearly a fourfold increase. Similar increases are observed for signing and verification operations, indicating that all ECC operations scale with curve size.

This increase is primarily due to the higher computational cost associated with processing larger integers. ECC operations rely heavily on scalar multiplication, which involves repeated point addition and doubling. As the bit size increases, the number of required arithmetic operations also increases, resulting in longer execution times.

Verification consistently requires the highest number of cycles, followed by signing and key generation. This indicates that verification is the most computationally intensive operation, as it involves multiple scalar multiplications and additional validation steps. Overall, these results demonstrate that while larger curves provide stronger security, they introduce a significant performance overhead.

### B. Latency Analysis

Latency increases significantly as key size increases, as shown in Fig. 4. The secp256r1 curve exhibits the highest latency due to increased arithmetic complexity.

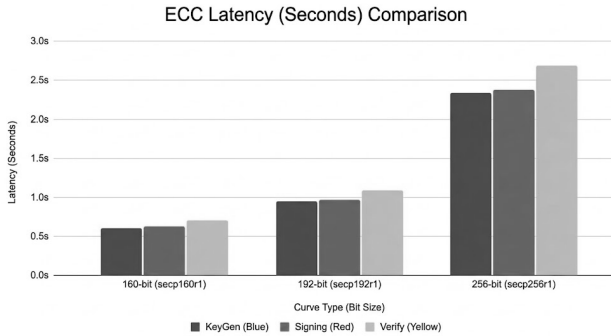


Fig. 4. ECC latency comparison across key sizes.

### C. Throughput Analysis

Throughput decreases as key size increases, demonstrating an inverse relationship with execution time. The smallest curve achieves the highest throughput, while the largest curve performs the worst.

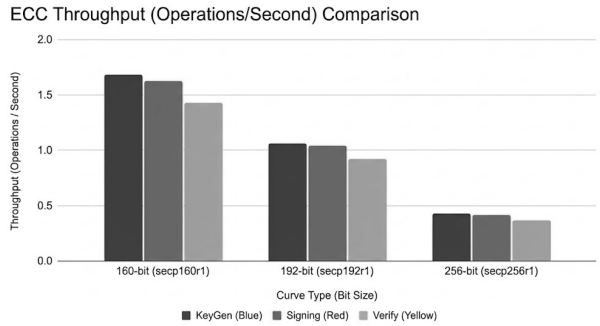


Fig. 5. ECC throughput comparison across key sizes.

### D. Output Verification

The UART output confirms correct execution of ECC operations and consistent measurement of cycle counts.

### E. Performance Trade-offs

The results demonstrate a clear trade-off between security and performance. Smaller curves provide faster execution and higher throughput but offer lower security. Larger curves provide stronger security but introduce significantly higher computational cost.

## V. DISCUSSION AND FUTURE WORK

The results of this study highlight both the strengths and limitations of implementing Elliptic Curve Cryptography (ECC) on a softcore processor without hardware acceleration. While ECC provides strong security with relatively small key sizes, the performance overhead becomes increasingly significant as curve size grows. This is especially evident in verification operations, which consistently exhibit the highest computational cost due to repeated scalar multiplications and additional validation steps.

One key observation is that the MicroBlaze processor executes all ECC operations sequentially in software, which limits overall efficiency. Unlike dedicated cryptographic hardware, the absence of parallelism and specialized arithmetic units results in increased latency and reduced throughput, particularly for larger key sizes such as secp256r1. This makes purely software-based ECC implementations less suitable for real-time or high-throughput embedded applications.

Future work can focus on improving performance through hardware acceleration. Implementing critical ECC operations, such as modular multiplication or point addition, as custom hardware modules on the FPGA could significantly reduce execution time. Additionally, exploring parallel processing techniques or pipelined architectures may help improve throughput without sacrificing security.

Another potential direction is optimizing the software implementation itself. This includes using more efficient algorithms for scalar multiplication, such as windowing methods or Montgomery ladder techniques, which can reduce the total number of operations required. Evaluating energy consumption alongside performance metrics would also provide a more comprehensive understanding of ECC suitability for resource-constrained systems.

## VI. LIMITATIONS

This study is subject to several limitations related to the hardware and implementation approach. First, all ECC operations were executed entirely in software on a 32-bit MicroBlaze processor without any hardware acceleration. As a result, the measured performance reflects a worst-case scenario for ECC execution rather than an optimized implementation.

Additionally, the system operates under resource constraints typical of embedded environments, including limited clock speed, memory availability, and lack of parallel processing capabilities. These constraints impact execution time and throughput, particularly for larger key sizes such as secp256r1.

Another limitation is that the benchmarking focuses primarily on execution cycles, latency, and throughput, without considering power consumption or energy efficiency. In embedded and IoT systems, energy usage

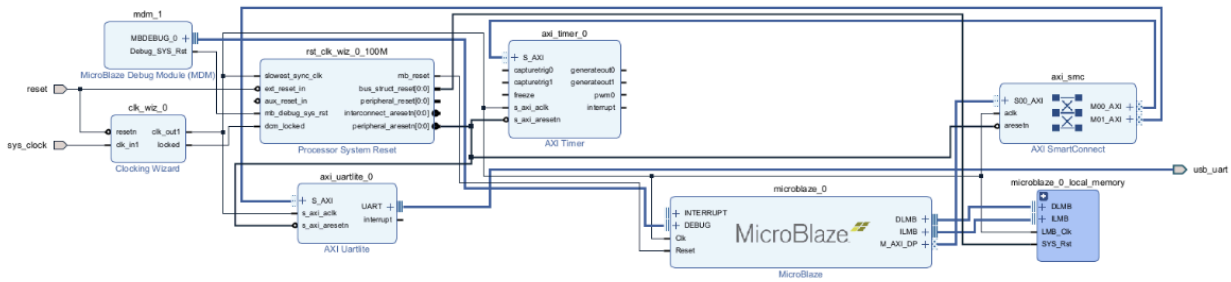


*Fig. 2. MicroBlaze processor system architecture.*

is often a critical factor, and future evaluations should incorporate these metrics for a more comprehensive analysis.

Finally, the results are based on a fixed hardware configuration using a Nexys A-7 FPGA. While this allows for controlled comparison across different curve sizes, the findings may not directly generalize to other processor architectures or hardware platforms.

## VII. CONCLUSION

This study demonstrates the performance trade-offs of scaling Elliptic Curve Cryptography (ECC) across 160-bit, 192-bit, and 256-bit key sizes on a 32-bit MicroBlaze softcore processor. By executing ECC entirely in software on a Nexys A-7 FPGA, the hardware configuration remained constant, allowing performance differences to be directly attributed to the increased computational complexity of larger curves. The results show a clear relationship between key size and system performance: as curve size increases, latency and cycle counts increase, while throughput decreases. This behavior is driven by the higher computational cost of scalar multiplication and modular arithmetic, with verification consistently being the most expensive operation.

The findings also highlight the limitations of running ECC on a softcore processor without hardware acceleration, as all operations are performed sequentially in software. Smaller curves provide better efficiency for resource-constrained systems, while larger curves offer stronger security at the cost of increased overhead. Therefore, selecting an appropriate ECC curve requires balancing performance and security requirements. Future work may focus on hardware acceleration or optimized implementations to improve efficiency while maintaining strong security.

## REFERENCES

[1] N. Koblitz, “Elliptic Curve Cryptosystems,” Mathematics of Computation, vol. 48, no. 177, pp. 203–209, 1987.

[2] V. S. Miller, “Use of Elliptic Curves in Cryptography,” Advances in Cryptology—CRYPTO ’85, Lecture Notes in Computer Science, vol. 218, pp. 417–426, 1986.

[3] Standards for Efficient Cryptography Group (SECG), “SEC 2: Recommended Elliptic Curve Domain Parameters,” 2010.

[4] Nexys A-7 100-T, “MicroBlaze Processor Reference Guide,” UG984, 2023.

[5] K. Osvik, A. Bos, D. Stefan, and D. Canright, “Fast Software Implementation of ECC on Embedded Systems,” Journal of Cryptographic Engineering, 2010.